

CAHIER DES CHARGES

Application Desktop de Traçabilité Commerciale

Gestion et suivi des documents Sage — Application Windows (.exe)

Domaine Gestion commerciale

Stack React · Spring Boot · PostgreSQL · PyInstaller

Table des matières

1	Introduction	3
1.1	Contexte et problématique	3
1.2	Objectifs du projet	3
1.3	Portée du document	4
2	Processus Métier	5
2.1	Cycle de vie d'une transaction	5
2.2	Règles de transition entre étapes	5
2.3	Relations entre entités	5
3	Spécifications Fonctionnelles	6
3.1	Tableau de bord principal	6
3.1.1	Indicateurs clés de performance (KPIs)	6
3.1.2	Graphiques	6
3.2	Gestion des clients et sociétés	7
3.2.1	Client (personne physique)	7
3.2.2	Société (personne morale)	7
3.2.3	Historique par entité	7
3.3	Module Devis	7
3.3.1	Tableau de bord du module	7
3.3.2	Création d'un devis	7
3.4	Module Bon de Commande (BDC)	8
3.5	Module Bon de Livraison (BL)	8
3.6	Module Attachement	9
3.6.1	Description	9
3.7	Module Facture	9
3.8	Module Encaissement	10
3.9	Module Traçabilité (fonctionnalité principale)	11
3.9.1	Description	11
3.9.2	Interactions	11
3.10	Vue Timeline Interactive	11
3.11	Recherche et Filtres	11
3.11.1	Recherche avancée	11

3.11.2 Filtres disponibles	11
3.12 Notifications Automatiques	11
3.13 Gestion Documentaire	12
3.14 Gestion des Utilisateurs	12
4 Architecture Technique	13
4.1 Architecture des données	13
4.2 Stack technique recommandée	13
4.3 Principes d'architecture	13
5 Conclusion	15

1. Introduction

1.1 Contexte et problématique

L'entreprise utilise actuellement le logiciel **Sage** pour générer l'ensemble de ses documents commerciaux : devis, bons de commande (BDC), bons de livraison (BL) et factures. Bien que Sage assure la production de ces documents, le suivi inter-documents reste insuffisant et les relations entre eux peu visibles pour les équipes.

Cette situation génère plusieurs difficultés opérationnelles :

- Absence de vue consolidée du cycle de vie d'une transaction ;
- Risque de perte d'informations lors du passage d'une étape à l'autre ;
- Difficulté à identifier rapidement les dossiers bloqués ou en retard ;
- Suivi financier manuel et source d'erreurs.

Périmètre fonctionnel : L'application n'a *pas* pour vocation de remplacer ou de dupliquer Sage. Elle importe les documents produits par Sage et crée des liens dynamiques entre eux afin d'en assurer la traçabilité complète. L'application est distribuée sous forme d'un **exécutable Windows (.exe)**, généré via **PyInstaller** à partir d'un script launcher Python qui orchestre le démarrage de l'interface React.

1.2 Objectifs du projet

Le projet vise à développer une **application desktop Windows** dont l'interface est construite en React et packagée sous forme d'un **exécutable natif (.exe)** via **PyInstaller** (script launcher Python), permettant de :

1. **Centraliser** tous les documents commerciaux issus de Sage ;
2. **Assurer la traçabilité** complète du cycle Devis → BDC → BL → Attachement → Facture → Paiement ;
3. **Visualiser** de façon dynamique le parcours d'une transaction ;
4. **Faciliter la recherche** par numéro, client, date, montant, statut ;
5. **Suivre l'état d'avancement** des dossiers en temps réel ;
6. **Digitaliser** le processus métier et réduire les pertes d'informations ;
7. **Améliorer le suivi financier** (encaissements, restes à payer, relances).

1.3 Portée du document

Ce cahier des charges couvre :

- La description du processus métier ;
- Les spécifications fonctionnelles détaillées de chaque module ;
- L'architecture des données et la stack technique recommandée.

2. Processus Métier

2.1 Cycle de vie d'une transaction

Chaque document commercial constitue une étape conditionnée par la validation de l'étape précédente. La chaîne globale est la suivante :



2.2 Règles de transition entre étapes

Étape source	Condition requise	Étape suivante
Devis	Statut = <i>Accepté</i>	Bon de commande (BDC)
BDC	Statut = <i>Validé</i>	Bon de livraison (BL)
BL	Statut = <i>Livré</i>	Attachement
Attachement	Statut = <i>Validé</i> (accord mutuel des deux parties)	Facture
Facture	Aucune (multiple paiements possibles)	Encaissement(s)

TABLE 2.1 – Règles de transition du cycle commercial

2.3 Relations entre entités

- **Client / Société** → 1 à N Devis
- **Devis** → exactement 1 BDC
- **BDC** → exactement 1 BL
- **BL** → exactement 1 Attachement
- **Attachement** → exactement 1 Facture
- **Facture** → 1 à N Paiements

3. Spécifications Fonctionnelles

3.1 Tableau de bord principal

Le tableau de bord constitue la page d'accueil de l'application et affiche des indicateurs en temps réel.

3.1.1 Indicateurs clés de performance (KPIs)

Indicateur	Description
Nombre de clients	Total des clients enregistrés
Nombre de sociétés	Total des sociétés enregistrées
Nombre de devis	Tous statuts confondus
Nombre de BDC	Tous statuts confondus
Nombre de BL	Tous statuts confondus
Nombre d'attachements	Tous statuts confondus
Nombre de factures	Tous statuts confondus
Montant total facturé	Cumul des montants facturés
Montant total encaissé	Cumul des paiements reçus
Montant restant à encaisser	Écart facturé/encaissé
Dossiers en cours	Transactions non clôturées

TABLE 3.1 – KPIs du tableau de bord

3.1.2 Graphiques

- Évolution mensuelle du nombre de devis ;
- Évolution mensuelle du nombre de factures ;
- Répartition des documents par statut (camembert) ;
- Comparaison montants encaissés vs restants à encaisser (barres).

3.2 Gestion des clients et sociétés

L'application distingue deux types d'entités commerciales indépendantes.

3.2.1 Client (personne physique)

- Nom, prénom
- Adresse e-mail, numéro de téléphone
- Adresse postale
- Matricule fiscal
- Observations libres

3.2.2 Société (personne morale)

- Raison sociale
- Responsable (nom du contact principal)
- Adresse, téléphone, e-mail
- Matricule fiscal
- Numéro de registre de commerce
- Observations libres

3.2.3 Historique par entité

Chaque fiche client ou société dispose d'un onglet **Historique** affichant : devis, BDC, BL, attachements, factures et paiements associés, classés par ordre chronologique.

3.3 Module Devis

3.3.1 Tableau de bord du module

Affichage de compteurs filtrables : *Tous / Acceptés / Refusés / En attente*.

3.3.2 Création d'un devis

Un bouton + **Nouveau devis** permet de créer un devis en associant :

- Un client existant, ou
- Un nouveau client, ou
- Une nouvelle société.

Champs du formulaire :

Champ	Type	Valeurs / Contraintes
Numéro de devis	Texte	Unique, format libre
Date	Date	Obligatoire
Date d'expiration	Date	Postérieure à la date de création
Description	Texte long	Obligatoire
Montant	Numérique	> 0
Statut	Enum	En attente / Accepté / Refusé
Commentaires	Texte long	Optionnel
Pièce jointe	PDF	Document Sage correspondant

TABLE 3.2 – Champs du module Devis

3.4 Module Bon de Commande (BDC)

Pré-condition : Le devis lié doit avoir le statut *Accepté*.

Champ	Type	Valeurs / Contraintes
Numéro BDC	Texte	Unique
Date	Date	Obligatoire
Description	Texte long	Obligatoire
Montant	Numérique	> 0
Statut	Enum	En préparation / Validé / Annulé
Référence devis	Lien	Devis source obligatoire
Pièce jointe	PDF	Document Sage correspondant

TABLE 3.3 – Champs du module BDC

Relation : 1 Devis $\rightarrow$ 1 BDC (relation one-to-one).

3.5 Module Bon de Livraison (BL)

Pré-condition : Le BDC lié doit avoir le statut *Validé*.

Les champs sont similaires au BDC avec l'ajout d'une référence au BDC source.

Statuts disponibles : En préparation / Livré / Partiellement livré / Annulé.

Relation : 1 BDC $\rightarrow$ 1 BL (relation one-to-one).

3.6 Module Attachement

Pré-condition : Le BL lié doit avoir le statut *Livré*.

3.6.1 Description

L'**attachement** est l'étape d'accord mutuel entre l'entreprise et son client (ou son fournisseur) qui précède la facturation. Il formalise la reconnaissance contradictoire des quantités, prestations ou montants effectivement livrés/réalisés, avant l'émission de la facture correspondante. Tant que l'attachement n'est pas validé par les deux parties, la facture ne peut pas être créée.

Champ	Type	Valeurs / Contraintes
Numéro d'attachement	Texte	Unique
Date	Date	Obligatoire
Description	Texte long	Obligatoire (quantités/prestations attachées)
Montant attaché	Numérique	> 0
Statut	Enum	En attente d'accord / Validé / Contesté / Refusé
Date d'accord mutuel	Date	Renseignée à la validation par les deux parties
Référence BL	Lien	BL source obligatoire
Commentaires	Texte long	Optionnel
Pièce jointe	PDF	Document Sage / procès-verbal d'attachement signé

TABLE 3.4 – Champs du module Attachement

Relation : 1 BL $\rightarrow$ 1 Attachement (relation one-to-one).

3.7 Module Facture

Pré-condition : L'attachement lié doit avoir le statut *Validé*.

Champ	Type	Valeurs / Contraintes
Numéro facture	Texte	Unique
Date	Date	Obligatoire
Description	Texte long	Obligatoire
Montant total	Numérique	> 0
Statut	Enum	Non payée / Partiellement payée / Payée / En retard
Référence attachement	Lien	Attachement source obligatoire
Commentaires	Texte long	Optionnel
Pièce jointe	PDF	Document Sage correspondant

TABLE 3.5 – Champs du module Facture

Relation : 1 Attachement $\rightarrow$ 1 Facture (relation one-to-one).

3.8 Module Encaissement

Une facture peut recevoir **plusieurs paiements partiels ou complets**. Chaque paiement est enregistré avec les informations suivantes :

Champ	Type	Valeurs / Contraintes
Référence paiement	Texte	Unique
Date	Date	Obligatoire
Montant reçu	Numérique	> 0
Mode de paiement	Enum	Espèces / Chèque / Virement bancaire / Carte bancaire
Référence bancaire	Texte	Optionnel (chèque, virement)
Commentaire	Texte long	Optionnel
Pièces jointes	Fichier(s)	Justificatifs de paiement

TABLE 3.6 – Champs du module Encaissement

3.9 Module Traçabilité (fonctionnalité principale)

Ce module constitue le **cœur différenciateur** de l'application.

3.9.1 Description

Il présente une vue dynamique de type *roadmap* représentant l'ensemble du parcours d'une transaction :

Client → Devis DV-001 → BDC BDC-001 → BL BL-001 → Attachement ATT-001 →
Facture FAC-001 → Paiement 1 → Paiement 2

3.9.2 Interactions

Un clic sur n'importe quel nœud de la chaîne ouvre un panneau de détail affichant :

- Les informations complètes du document ;
- Le statut courant ;
- Les montants associés ;
- Le ou les documents PDF liés ;
- Les documents amont et aval.

3.10 Vue Timeline Interactive

Affichage chronologique de tous les événements liés à une transaction : création, acceptation, validation, livraison, attachement, facturation, paiements reçus, etc.

3.11 Recherche et Filtres

3.11.1 Recherche avancée

Recherche multi-critères sur : numéro de document, client, société, date, montant.

3.11.2 Filtres disponibles

- **Statut** : filtre par valeur d'état ;
- **Période** : aujourd'hui / semaine / mois / année / personnalisée ;
- **Montants** : fourchette min/max.

3.12 Notifications Automatiques

Le système génère des alertes automatiques pour les situations suivantes :

- Devis en attente depuis plus de **7 jours** ;

- Attachements en attente d'accord mutuel depuis plus de **7 jours** ;
- Factures ayant le statut *En retard* ;
- Paiements partiels avec solde restant significatif ;
- Documents manquants dans la chaîne (ex. : BDC sans BL associé, BL sans attachement associé).

3.13 Gestion Documentaire

Stockage sécurisé des PDFs et autres fichiers avec les métadonnées suivantes :

- Nom du fichier ;
- Date d'import ;
- Type de document ;
- Taille du fichier ;
- Utilisateur ayant importé le fichier.

3.14 Gestion des Utilisateurs

Rôle	Périmètre d'accès
Administrateur	Accès total : gestion des utilisateurs, configuration, tous les modules
Comptable	Gestion financière : attachements, factures, encaissements, suivi des paiements
Commercial	Gestion des devis et du suivi client
Lecture seule	Consultation sans modification

TABLE 3.7 – Rôles et droits d'accès

4. Architecture Technique

4.1 Architecture des données

Le modèle de données reflète les relations métier décrites au chapitre 2 :

- **Client / Société** → 1 à N **Devis**
- **Devis** → 1 **BDC**
- **BDC** → 1 **BL**
- **BL** → 1 **Attachement**
- **Attachement** → 1 **Facture**
- **Facture** → 1 à N **Paielements**

4.2 Stack technique recommandée

Couche	Technologie
Frontend	React 18+ · Vite · TypeScript · Material UI (MUI)
Launcher	Script Python (<code>launcher.py</code>) orchestrant le démarrage de l'app
Packaging	PyInstaller — génération d'un <code>.exe</code> Windows autonome
Commande build	<code>pyinstaller --onefile --windowed --icon=logo.ico launcher.py</code>
Backend	Spring Boot (Java 17+) · API REST
Base de données	PostgreSQL 15+
Authentification	JSON Web Token (JWT)
Stockage fichiers	Système de fichiers local (machine utilisateur)

TABLE 4.1 – Stack technique recommandée

4.3 Principes d'architecture

- **Launcher Python** : un script `launcher.py` assure le démarrage coordonné du serveur backend (Spring Boot) et de l'interface frontend (React/Vite), puis ouvre

- l'application dans une fenêtre native ;
- **Packaging PyInstaller** : la commande `pyinstaller --onefile --windowed --icon=logo.ico launcher.py` compile le launcher et toutes ses dépendances en un seul exécutable `.exe` autonome, distribuable sans installation Python sur le poste cible ;
- **Séparation frontend/backend** via une API REST documentée (OpenAPI/Swagger) ;
- **Authentification stateless** par JWT avec gestion des rôles côté backend ;
- **Validation des règles métier** (conditions de transition, y compris l'accord mutuel d'attachement) appliquée côté serveur ;
- **Stockage des fichiers** sur le système de fichiers local avec référencement en base de données ;
- **Pagination et filtres** pris en charge par le backend pour les listes volumineuses.

5. Conclusion

Ce projet vise à transformer des documents Sage aujourd'hui isolés en un **parcours métier complet, visuel, dynamique et traçable de bout en bout**, distribué sous la forme d'une **application desktop Windows (.exe)**.

L'interface React est packagée via un script launcher Python compilé avec **PyInstaller**, offrant un exécutable autonome installable sur n'importe quel poste Windows sans prérequis technique. L'application ne se substitue pas à Sage mais y ajoute une couche de **liaison et d'intelligence**, offrant aux équipes une visibilité totale sur chaque transaction commerciale, depuis l'émission du devis jusqu'au dernier paiement encaissé, en passant par l'étape clé de l'**attachement** qui formalise l'accord mutuel entre les parties avant facturation.